UNIVERSIDADE
LUSÓFONA

Making Of

Trabalho realizado por:

Rodrigo Beleza a22407392

Índice

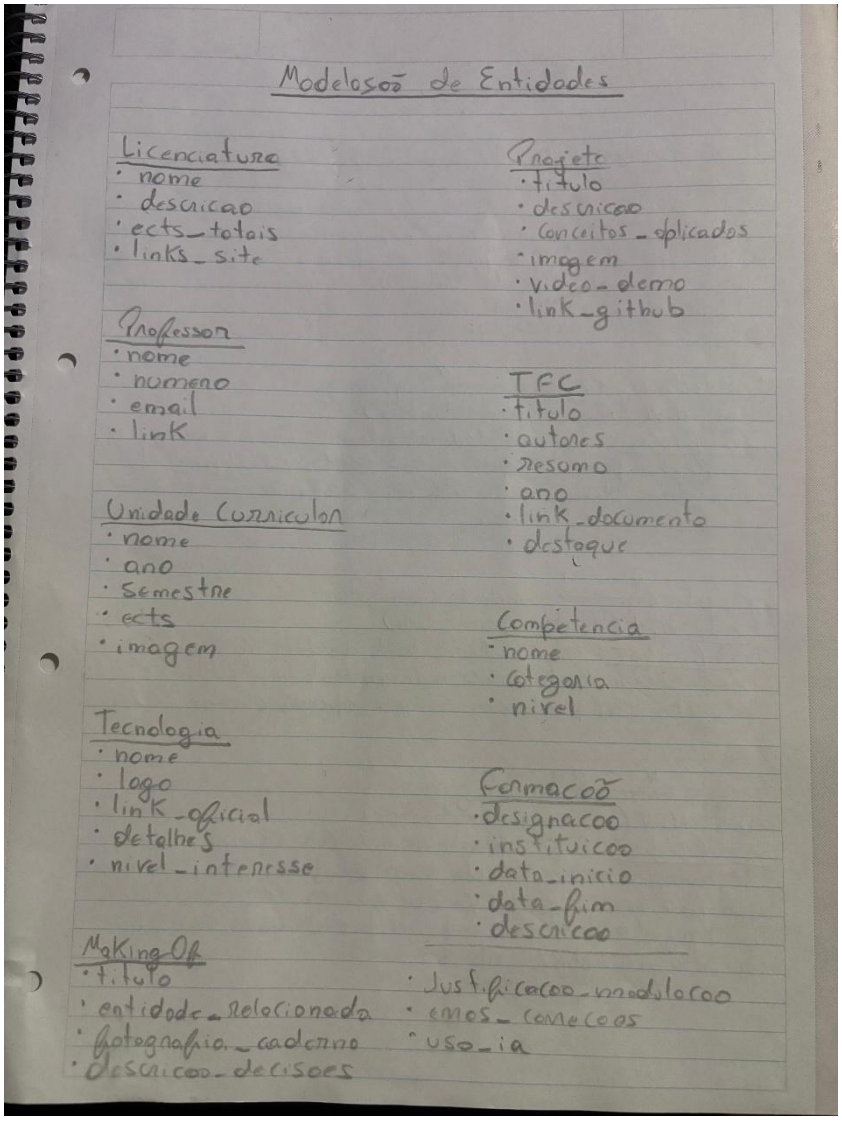
Introdução 3
Entidades 3
Diagrama 5
Relações 7
Problemas e Soluções 8
Integração de Dados e Automação via JSON 9
Conclusão 10

Introdução

O presente documento, designado por *Making Of*, reporta o processo de conceção, modelação e desenvolvimento de uma aplicação web destinada a servir como portfólio académico e pessoal. Este projeto foi desenvolvido no âmbito da unidade curricular de Programação Web, integrada na Licenciatura de Informática de Gestão da Universidade Lusófona.

Mais do que um simples manual de utilizador do produto final, este relatório foca-se nos "bastidores" da construção da aplicação. O desenvolvimento de software robusto exige um planeamento rigoroso e, como tal, o objetivo principal deste documento é expor e justificar a linha de raciocínio lógico adotada durante a fase de arquitetura de dados, refletindo a transição do pensamento analógico (no papel) para a implementação digital (em *Django*).

Entidades



Modelosoõ de Entidades

Licenciatura
- nome
- descricao
- ects_totais
- links_site

Professor
- nome
- numero
- email
- link

Unidade Curricular
- nome
- ano
- semestre
- ects
- imagem

Tecnologia
- nome
- logo
- link_oficial
- detalhes
- nivel_interesse

MaKing Of
- titulo
- entidade_relacionada
- fotografia_caderno
- descricoo_decisoes

Projeto
- titulo
- descricoo
- conceitos_aplicados
- imagem
- video_demo
- link_github

TFC
- titulo
- autores
- resumo
- ano
- link_documento
- destaque

Competencia
- nome
- categoria
- nivel

Formacoõ
- designacoo
- instituicoo
- data_inicio
- data_fim
- descricoo

- Justificacoo_modulacoo
- emos_comecos
- uso_ia

Implementação da entidade Professor

Numa fase inicial de abstração, a associação dos docentes às disciplinas poderia ser pensada como um simples campo de texto (ex: docente = CharField) dentro da entidade UnidadeCurricular. No entanto, essa abordagem violaria as regras de normalização de bases de dados. Ao isolar o "Professor" numa entidade própria, evitou-se a duplicação de dados (o mesmo professor lecionar várias cadeiras não obriga a repetir a inserção dos seus dados de contacto) e preparou-se o sistema para escalar. Para além disso, considero ser uma entidade importante, poque é bom referir as informações de quem trabalhámos para o portefólio estar mais claro e organizado.

Justificação dos atributos da entidade Professor

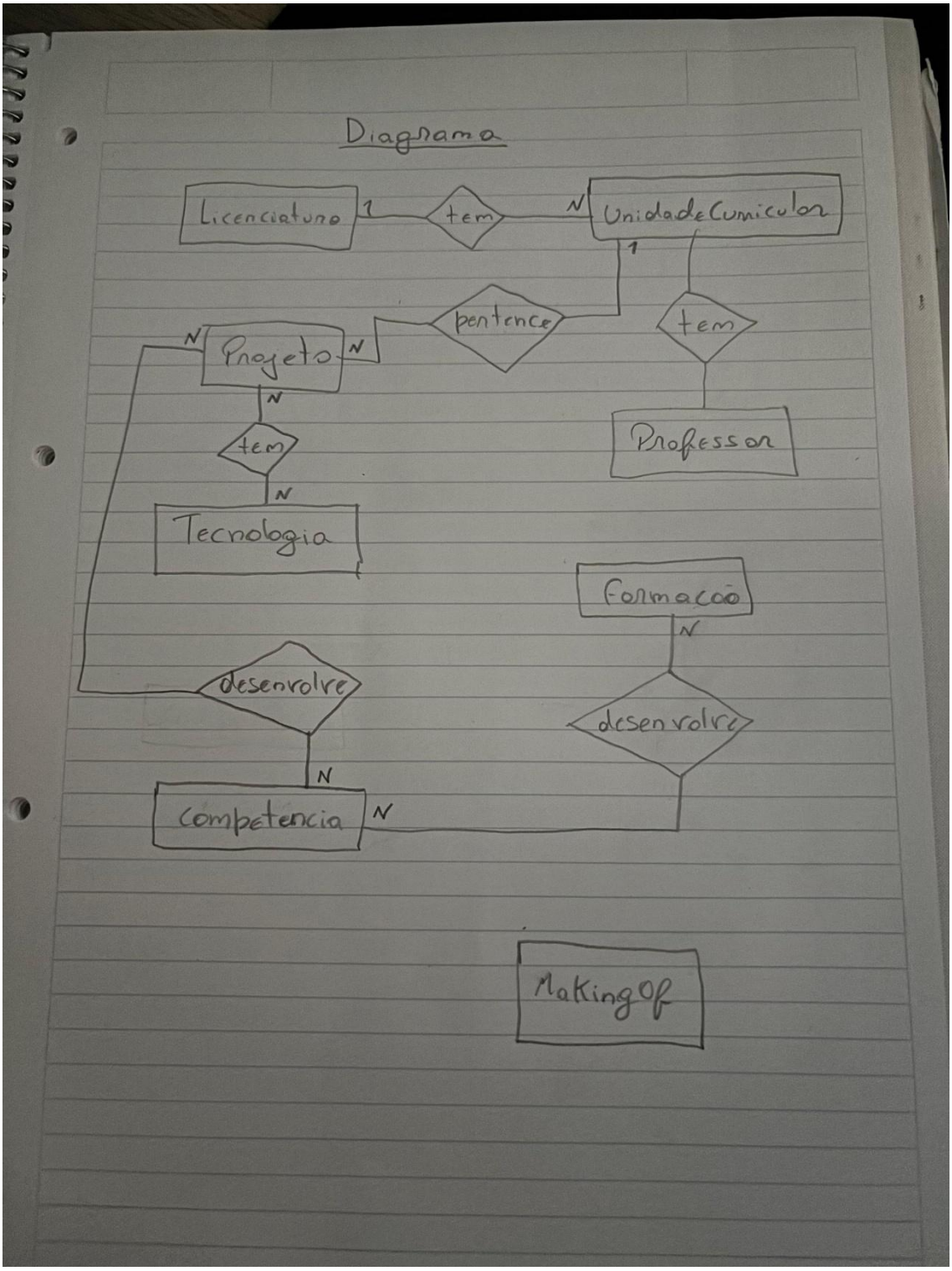
nome (CharField): Atributo base para a identificação do professor.

numero (CharField - Obrigatório): O número de docente atua como um identificador único institucional. Durante o desenvolvimento, este campo foi deliberadamente configurado como obrigatório (removendo as propriedades blank=True e null=True), de modo a garantir a consistência da base de dados e evitar a ambiguidade em casos de professores com nomes homónimos. Para além disso, é um identificador essencial dos professoresd na faculade.

email (EmailField): Essencial para efeitos de contacto. A utilização do tipo EmailField delega no Django a responsabilidade de validar automaticamente a formatação do endereço, prevenindo erros de digitação por parte do administrador.

link_lusofona (URLField): Permite associar o docente à sua página oficial no website da Universidade. Este detalhe acrescenta credibilidade ao portfólio, permitindo ao utilizador final validar facilmente o corpo docente que acompanhou o percurso académico do aluno.

Diagrama



A entidade MakingOf foi propositadamente mantida sem relações diretas (Foreign Keys) no diagrama pelas seguintes razões arquiteturais:

- **Âmbito Global:** O Making Of serve como um diário de bordo para todo o sistema e ligar esta entidade a apenas uma tabela limitaria a sua utilidade.
- **Prevenção de Campos Vazios (Nulls):** Criar múltiplas chaves estrangeiras para permitir ligar o Making Of a qualquer entidade do projeto geraria demasiados campos vazios (NULL) a cada novo registo, o que é uma má prática de modelação.
- **Preservação Histórica:** O histórico de desenvolvimento deve ser permanente. Ao não ter ligações estritas, garantimos que se um "Projeto" for apagado no futuro, a documentação do esforço técnico envolvido na sua criação não se perde.

Relações

Relações

Licenciatura $\overset{1}{\longleftrightarrow}$ $\overset{N}{}$ Unidade Curricular

Tipo: 1:N (um-para-Muitos)

Justificação: Uma licenciatura pode ter vários cadeiras e cada cadeira vai pertencer a uma licenciatura. Optei por dizer que pertence apenas a uma licenciatura pois no contexto do trabalho só falamos na minha licenciatura e não em mais.

Unidade Curricular $\overset{N}{\longleftrightarrow}$ $\overset{N}{}$ Professor

Tipo: N:N (Muitos-para-Muitos)

Justificação: Na realidade académica que vivemos, um professor pode lecionar várias cadeiras diferentes. Para além disso, a mesma cadeira pode ter vários professores a lecionar.

Projeto $\overset{N}{\longleftrightarrow}$ $\overset{1}{}$ Unidade Curricular

Tipo: N:1 (Muitos-para-um)

Justificação: Uma cadeira pode pedir vários projetos para serem feitos, mas independentemente disso, esse projeto vai pertencer sempre apenas a essa cadeira.

Projeto $\overset{N}{\longleftrightarrow}$ $\overset{N}{}$ Tecnologia

Tipo: N:N (Muitos-para-Muitos)

Justificação: Dificilmente vamos ver apenas uma tecnologia a ser usada num projeto, por isso, achei que fez mais sentido fazer assim. Por outro lado, um projeto também em princípio vai usar várias tecnologias diferentes.

Relações (continuação)

Projeto $\overset{N}{\longleftrightarrow}$ Competencia
Tipo: N:N (Muitos-para-Muitos)
Justificação: Os projetos que fazemos são muito importantes para o desenvolvimento das nossas competências. Elas permitem aprender e desenvolver muitas coisas, por isso obtei pela relação muitos-para-muitos. Para além disso, podemos desenvolver as mesmas competências em projetos diferentes.

Formação $\overset{N}{\longleftrightarrow}$ Competência
Tipo: N:N (Muitos-para-muitos)
Justificação: Quando nos formamos numa determinada área nós desenvolvemos uma série de competências diferentes. Para além disso, também podemos desenvolver a mesma competência em vários cursos diferentes.

MakingOf $\longleftrightarrow$ (Todas as outras entidades)
Justificação: Sendo a entidade que relata o que acontece em todo o projeto tem que ter algum tipo de ligação com todos. Criei um campo que vai categorizar a tabela, para não ter que ligar diretamente a todos.

Problemas e Soluções

1º - Problema de relações com a entidade MakingOf

Problema: Ao tentar relacionar o MakingOf com as outras entidades, percebi que seria muito complicado porque teria muitos campos diferentes um para cada uma

das outras entidades. Tendo isto em conta, iria tornar-se muito difícil de visualizar e analisar tanto no programa, como no diagrama.

Solução: Resolvi criar um atributo de classificação ('entidade_relacionada') que vai ser no fundo uma etiqueta. Esta etiqueta vai ajudar-me a definir a que entidade pertence o registo, evitando assim campos nulos e com muita informação desnecessária.

2º - Problema na normalização dos dados entre Professores e Unidades Curriculares

Problema: Durante a modelação em papel, surgiu a dúvida sobre como associar os docentes às disciplinas. Se adicionasse apenas um campo de texto "Professor" na Unidade Curricular, não conseguiria associar múltiplos docentes à mesma cadeira sem criar texto livre desestruturado. Se usasse uma relação de 1 para Muitos, teria de repetir o registo do mesmo professor sempre que este lecionasse mais do que uma disciplina, violando os princípios de normalização de bases de dados.

Solução: Criei uma entidade autónoma Professor e estabeleci uma relação de Muitos-para-Muitos na entidade UnidadeCurricular. Esta solução reflete a realidade académica, ou seja, um professor ensina várias UCs, e uma UC tem vários professores, evitando duplicação de dados e permitindo adicionar informações extra ao docente, como o número de professor e o e-mail.

3º - Duplicação de Dados em Execuções Sucessivas do Script

Problema: Durante a fase de testes do carregamento de dados pelo terminal precisei de executar o script loader.py várias vezes para testar diferentes versões do JSON. Isto resultou num problema de duplicação, ou seja, a cada execução, o Django adicionava os mesmos TFCs repetidamente à base de dados, poluindo a tabela.

Solução: Implementei uma instrução de purga no início do script de importação. Ao invocar o comando TFC.objects.all().delete() imediatamente antes de ler o ficheiro JSON, garanto que a tabela é limpa de dados antigos ou testes anteriores. Isto assegura que a base de dados reflete sempre o estado exato e atualizado do ficheiro JSON.

Integração de Dados e Automação via JSON

Para garantir que a plataforma fosse não apenas um modelo estrutural vazio, mas um sistema funcional alimentado com dados reais e consistentes, implementei

duas rotinas distintas de importação automatizada baseadas em ficheiros JSON. Esta decisão arquitetural visou eliminar a necessidade de inserção manual.

1 - Carregamento de dados dos TFCs

A primeira fase consistiu no carregamento do histórico de Trabalhos Finais de Curso (TFCs). Para o efeito, desenvolvi um script de importação local (loader.py) com as seguintes características técnicas:

Idempotência e Prevenção de Duplicados: O script foi desenhado para iniciar com uma rotina de purga (TFC.objects.all().delete()), garantindo que, independentemente do número de execuções, a base de dados não acumula registos duplicados, espelhando exatamente o conteúdo do ficheiro de origem.

Tolerância a Falhas na Estrutura JSON: Sabendo que os ficheiros JSON podem apresentar inconsistências estruturais utilizei o método .get() do Python. Isto permitiu salvaguardar a execução do código contra erros fatais, atribuindo valores nulos ou vazios por defeito caso a informação não existisse, mantendo a integridade do processo de importação.

2 - Consumo de API Externa e Tratamento de Dados (Universidade Lusófona)

O desafio de maior complexidade técnica envolveu a extração e integração do plano de estudos oficial da Universidade Lusófona, consumindo uma API pública para preencher automaticamente as entidades Licenciatura e UnidadeCurricular.

Esta fase exigiu a resolução de vários obstáculos do mundo real:

Mapeamento Relacional Dinâmico: No script de carregamento (loader_lusofona.py), garanti o respeito absoluto pela cardinalidade da base de dados. O código cria, primeiramente, a entidade "Licenciatura" e, durante a iteração do plano de estudos, associa dinamicamente cada nova "Unidade Curricular" a essa licenciatura através de Foreign Keys, automatizando por completo o encadeamento relacional.

Erro no import dos dados: O desafio final surgiu perante a presença de "dados sujos" (dirty data) na resposta da API. O modelo de dados exigia um valor numérico inteiro (IntegerField) para o atributo semestre, mas a API retornava pontualmente strings textuais (ex: "Semestral"). Então resolvi alterar o tipo do parâmetro na base de dados.

Conclusão

O desenvolvimento deste portfólio académico em Django representou uma etapa fundamental na minha consolidação de competências em desenvolvimento Web e Engenharia de Software. Mais do que a simples criação de uma montra digital

para os meus projetos, este desafio exigiu a conceção de um sistema robusto, escalável e alicerçado em boas práticas de arquitetura de software.

Ao longo do projeto, destaco a modelação rigorosa da base de dados relacional, onde a criação de entidades exigiu uma análise cuidada das cardinalidades e da normalização de dados. A decisão de documentar o processo de forma isolada na entidade Making Of demonstrou a importância de planear a arquitetura não apenas para o presente, mas para a preservação histórica do conhecimento técnico.

Em suma, este projeto cumpriu com distinção o seu objetivo. Resultou numa plataforma dinâmica, profissional e facilmente expansível, que não só reflete o meu percurso académico e técnico até ao momento, como também serve de prova viva das minhas capacidades enquanto programador capaz de resolver problemas complexos e arquitetar soluções eficientes.